01 - Count and accumulate

Explained answers and feedback

Trace a counted loop and distinguish a counter from a running total.

Textbook: loops

A

Rounds 1,2,3,4 add 2,4,6,8; running totals 2,6,12,20. Output 20.

B / exit

Use total = 0; for value in range(1, n + 1): total += value. Totals for 4,0,1 are 10,0,1. Zero is the additive identity; an empty range adds nothing. n = 1 exposes range(1,n): wrong 0 instead of 1.

Feedback

If the result is only the last value, locate accumulator initialisation. Retrace with n = 3 after the correction.

02 - Return a result from a function

Explained answers and feedback

Separate parameters, return values and printed output.

Textbook: functions

A

Returns 2 and 17. A print-only version displays a number but implicitly returns None; first would receive None.

B

def cover_cost(quantity, price): return quantity * price + 4. Returns 4,10,14. Accept non-negative integer quantity and non-negative numeric price, explicitly stated. Input validation is separate.

Exit / feedback

quantity and price are parameters; 3 and 2 are arguments in cover_cost(3,2). If print and return are confused, assign the result and inspect it before printing.

03 - Traverse a list and retain a position

Explained answers and feedback

Use zero-based indices and handle empty input deliberately.

Textbook: arrays

A

best remains 0 at index 1, changes to 2 at index 2, stays 2 at index 3. Output 2 2. With <= the last equal minimum wins: index 3.

B

Use if len(values) == 0 before any indexing; else execute the algorithm. Expected No data; 0 9; 1 1. For the last case the minimum value is 1 and its first index is 1.

Feedback

If indices start at 1, label positions 0..len-1 over the supplied values. Retest a single-element list and equal minima.

04 - Read a file and test bad records

Explained answers and feedback

Read sequential text, convert values and report unsuitable records.

Textbook: files

A

Prints Invalid: bad then Total: 5. -1 is convertible and is accepted by the original contract.

B

After value = int(text), use if 0 <= value <= 10: total += value; else: print("Out of range:", value). Test output: Out of range: 11; Out of range: -1; Invalid: oops; Total: 10. Blank is ignored.

Feedback / exit

No: open may raise FileNotFoundError before the inner try. A ValueError handler around conversion does not handle missing files. If 10 is rejected, check inclusive boundaries.